

Array

ma con le funzioni!

Perchè usare le funzioni:

Usare le funzioni ci permette di scrivere meno codice, riutilizzabile e di più facile lettura e correzione da eventuali errori. Tutta la programmazione moderna viene sviluppata per funzioni.

Lo sviluppo di funzioni prevede lo sviluppo della relativa **documentazione**, farla è un aiuto sia per noi stessi che per eventuali collaboratori al codice.

Recap su come lavorare per funzioni:

Lavorando per funzioni parte del codice viene spostato in un file di libreria, corredato del proprio file dei prototipi, quindi il nuovo asset sarà:

- **main program:** contiene il codice generale del programma, include il file `.c` della libreria
- **lib.h:** Contiene i prototipi delle funzioni e la relativa documentazione
- **lib.c:** Contiene la definizione delle funzioni, include il `.h`

Prima di convertire in funzioni ricorda:

- Il passaggio di un vettore ad una funzione è **sempre** per **riferimento**.
- E' buona norma passare anche la *DIM* del vettore, o il numero di celle da utilizzare ed è un passaggio per **valore**.
- La documentazione prima del prototipo è fondamentale *@param* e *@return* vanno specificati e compariranno nel suggerimento durante il richiamo nel main program.

Funzioni di base con Array

- Inizializzazione.
- Popolamento.
- Stampa.

Inizializzazione di un Array

Assegna ad ogni cella un valore predefinito iniziale. Viene richiamata subito dopo la *dichiarazione*.

```
// prototipo - lib.h
/** Azzera tutte le celle di un array.
 * @param int* Riferimento al vettore da inizializzare
 * @param int Dimensione dell'Array.
 */
void initArray(int _v[], int _dim);
```

```
// funzione - lib.c
void initArray(int _v[], int _dim){
    int i;
    for(i=0; i<_dim; i++){
        _v[i] = 0;
    }
}
```

_v e *_dim* sono riferimento e copia del vettore e della costante indicate in fase di chiamata.

A seguire vedrai il main program e sarà più chiaro.

Popolamento Manuale di un Array

Assegna ad ogni cella un valore richiesto in input all'utente.

```
// prototipo - lib.h  
/** Assegna ad ogni cella di un array un valore richiesto in input.  
 * @param int* Riferimento al vettore da acquisire.  
 * @param int Dimensione dell'Array.  
 */  
void manualInputArray(int _v[], int _dim);
```

```
// funzione -> lib.c
void manualInputArray(int _v[], int _dim){
    int i;
    char junk;

    for(i=0; i<_dim; i++){
        printf("Inserisci il [%d] valore: ", i+1);
        scanf("%d", &_v[i]);
        junk = getchar();
    }
}
```

Cambiando la *i* cambia la cella indicizzata che viene utilizzata.

Popolamento Random di un Array

Assegna ad ogni cella un valore random. (comodo per testing)

```
// prototipo - lib.h  
/** Assegna ad ogni cella di un array un valore random tra 1 e 99.  
 * @param int* Riferimento al vettore da riempire.  
 * @param int Dimensione dell'Array.  
 */  
void randomInputArray(int _v[], int _dim);
```

```
// ricordati di includere time.h e stdlib.h nel file lib.c  
// codice C popolamento automatico con random.  
void randomInputArray(int _v[], int _dim){  
    for(i=0; i<_dim; i++){  
        // genero altezze da 150 a 190 (intesi come centimetri)  
        _v[i] = 1 + (rand() % 99);  
    }  
}
```

Cambiando la *i* cambia la cella indicizzata che viene utilizzata.

Stampa valori su singola riga

```
// prototipo - lib.h
/** Stampa su singola riga i valori del vettore.
 * @param int* Riferimento al vettore da riempire.
 * @param int Dimensione dell'Array.
 */
void printRowArray(int _v[], int _dim);
```

```
// codice C stampa valori in riga.
void printRowArray(int _v[], int _dim){
    for(i=0; i<_dim; i++){
        printf("%d ", _v[i]);
    }
}
```

Ogni valore distanzia il successivo con uno spazio; vedi spazio vuoto dopo il *%d*.

Stampa valori in colonna con indice di cella.

```
// prototipo - lib.h
/** Stampa i valori dell'array in colonna con indice di cella.
 * @param int* Riferimento al vettore da riempire.
 * @param int Dimensione dell'Array.
 */
void printColArray(int _v[], int _dim);
```

```
// codice C stampa valori in colonna.
void printColArray(int _v[], int _dim){
    for(i=0; i<_dim; i++){
        printf("[%d]= %d", i, _v[i]);
        printf("\n");
    }
}
```

Ogni riga presenta tra parentesi quadre l'indice *i* in quel momento seguito dal contenuto della cella indicizzata. Ho separato l'andata a capo '*\n*' per comodità, ma poteva essere messa anche nella riga precedente.

Ecco come deve risultare il file *lib.h*

```
/** Azzera tutte le celle di un array.  
* @param int* Riferimento al vettore da inizializzare  
* @param int Dimensione dell'Array.  
*/  
  
void initArray(int _v[], int _dim);  
  
/** Assegna ad ogni cella di un array un valore random tra 1 e 99.  
* @param int* Riferimento al vettore da riempire.  
* @param int Dimensione dell'Array.  
*/  
void randomInputArray(int _v[], int _dim);  
  
// a seguire tutti gli altri prototipi e relativa documentazione...
```

Ecco come deve risultare il file *lib.c*

```
// includi tutte le librerie necessarie e il file lib.h dei prototipi.
#include <stdio.h>
#include <time.h>
#include <stdlib.h>
#include "lib.h"
void initArray(int _v[], int _dim){
    // tutto il codice della funzione
}
void randomInputArray(int _v[], int _dim){
    // tutto il codice della funzione
}
// a seguire tutte le altre funzioni...
```

Esempio di main e chiamate

```
// includi tutte le librerie necessarie e il file lib.h dei prototipi.
#include <stdio.h>
#include "lib.c"
#define DIM 10

int main(){
    int vet[DIM];           //dichiaro il vettore
    srand(time(NULL));      // per avere i numeri random nella relativa funzione.

    initArray(vet, DIM);     // richiamo l'inizializzazione
    printRowArray(vet, DIM); // richiamo la stampa su riga
    randomInputArray(vet, DIM); // richiamo input random del vettore
    printf("\n");
    printColArray(vet, DIM); // richiamo la stampa in colonna con indici

    return(0);
}
```

Let's try...

1. Crea un nuovo file *e02.c* (o altro numero per il main program)
2. Crea un nuovo file *array_lib.h* - per i prototipi
3. Crea un nuovo file *array_lib.c* - per le funzioni
4. Dichiara nel file *array_lib.h* i prototipi per le funzioni viste nella presentazione precedente. (Alcuni li hai già visti poco fa)
5. Definisci nel file *array_lib.c* tutte le funzioni con il loro codice necessario. (Alcune li hai già visti poco fa)

6. Nel main program, dichiara un vettore *vett* di 10 elementi definendo una *DIM*
7. Con apposita funzione inizializza il vettore (tutte le celle a 0).
8. Con apposita funzione procedi ad un input manuale dei valori nel vettore.
9. Con apposita funzione stampa tutti i valori in **colonna**.
10. Compila ed esegui il tuo codice *gcc file.c && ./a.out*

Take a break...